

# NumPy kutubxonasi haqida umumiy ma'lumot

5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.  
2-mashg'ulot. **NumPy va Pandas bilan ma'lumotlarni qayta ishlash** (Amaliy)

O'quv savollari:

1. NumPy kutubxonasida amallar bajarish
2. Pandas yordamida ma'lumotlarni qayta ishlash va tahlil qilish.

## 1-o'quv savoli: **NumPy kutubxonasida amallar bajarish**

### 1. NumPy nima?

**NumPy (Numerical Python)** — bu Python dasturlash tilida ilmiy va matematik hisob-kitoblar uchun eng asosiy kutubxonalardan biridir. NumPy massivlar (array) bilan ishlashni osonlashtiradi va an'anaviy Python ro'yxatlariga qaraganda tezroq, ixcham va samaraliroq hisoblanadi.

NumPy:

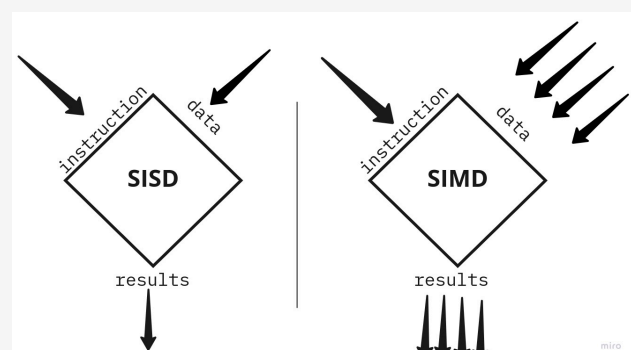
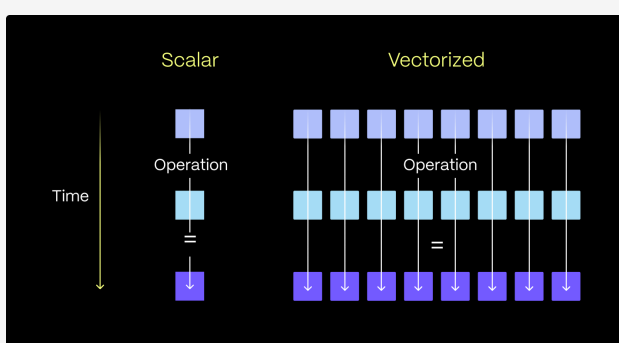
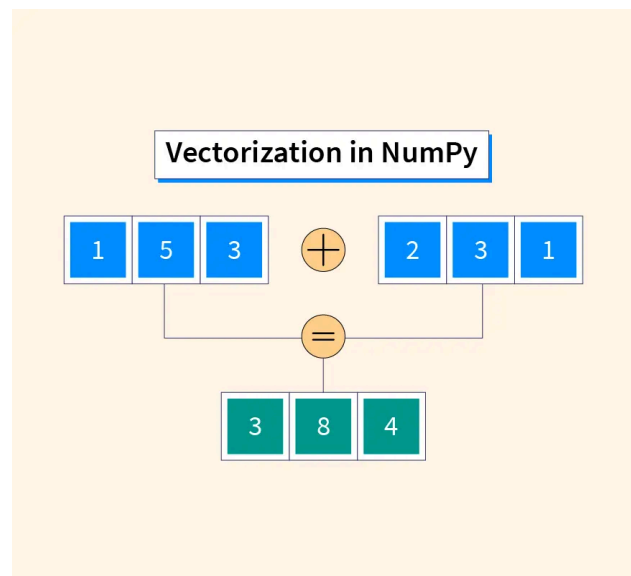
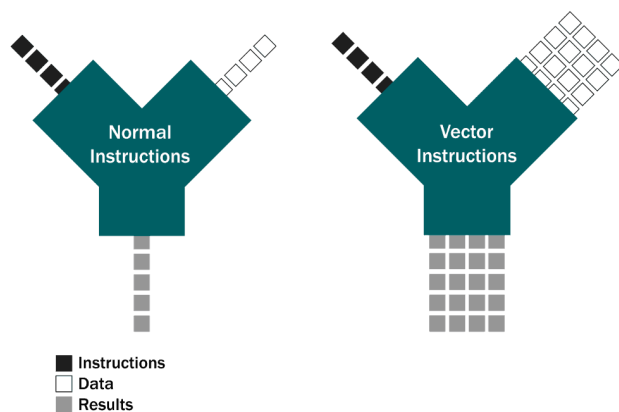
- katta hajmdagi sonli ma'lumotlar bilan ishlaydi,
- matritsa va vektorlar ustida amallarni bajaradi,
- statistik, matematik va chiziqli algebra funksiyalarini taqdim etadi,
- boshqa ML kutubxonalari (Pandas, Scikit-Learn, TensorFlow, PyTorch) uchun asos bo'lib xizmat qiladi.

NumPy amallari oddiy Python loqplari orqali emas, balki **C darajasida yozilgan vektorlashgan funksiyalar** orqali bajariladi.

Bu shuni anglatadi:

- Python har bir elementni for bilan aylanmaydi
- Birinchi darajada bitta buyruq barcha elementlarga qo'llanadi
- CPU'ning SIMD (Single Instruction Multiple Data) registrlari ishlatiladi

**Vektorlashgan hisoblash (vectorized computing)** – bu hisoblash jarayonini **elementma-element emas**, balki **bir nechta elementni bir vaqtning o'zida** (blok holida) bajaradigan optimallashtirilgan usuldir.



Sodda qilib aytganda:

**Oddiy usul:** bir xil operatsiyani 1 ta elementga **Vektorlashgan usul:** shu operatsiyani 1000 ta elementga *bir yo'la* bajaradi.

## Oddiy misol (Python – oddiy sikl)

```
In [1]: result = []
for x in [1, 2, 3, 4]:
    result.append(x * 2)
```

Bu yerda Python **har bir elementni alohida hisoblaydi** → sekinroq.

## Vektorlashgan misol (NumPy)

```
In [101... import numpy as np
```

```
In [92]: x = np.array([1, 2, 3, 4])
result = x * 2
```

Bu yerda NumPy **SIMD** (Single Instruction, Multiple Data) orqali → **barcha elementlarga bir yo'la** ko'paytirish bajaradi → **juda tez ishlaydi**.

# Vektorlashgan hisoblashning afzalliklari

| Afzallik                     | Izoh                                                |
|------------------------------|-----------------------------------------------------|
| <b>Tez ishlaydi</b>          | Bir nechta elementni bir vaqtning o'zida hisoblaydi |
| <b>Kam xotira sarfi</b>      | Loop overhead yo'q, CPU keshlari yaxshi ishlaydi    |
| <b>GPUga mos</b>             | GPUlar minglab operatsiyani parallel bajaradi       |
| <b>Kod qisqa va chiroyli</b> | "x * 2" kabi oddiy yoziladi                         |

## Xulosa

**Vektorlashgan hisoblash – bir xil operatsiyani ko'p elementlarga bir vaqtning o'zida bajarib, hisoblashni bir necha baravar tezlashtirish texnikasidir.**

Quyida, NumPy dagi **vectorization vs loop** bo'yicha benchmark (tezlik solishtiruv) kodi keltirilgan.

## 1-qism: Oddiy Python ro'yxati bilan hisoblash

```
In [93]: import time

# 10 millionta elementli ro'yxat
n = 10_000_000
py_list = list(range(n))

start = time.time()
result = [x * 2 for x in py_list]
end = time.time()

print("Oddiy Python ro'yxati uchun vaqt:", end - start, "sekund")
```

Oddiy Python ro'yxati uchun vaqt: 0.33017683029174805 sekund

Bu yerda:

- `py_list` – oddiy Python ro'yxat
- har bitta elementni `x * 2` qilib **list comprehension** orqali hisoblayapmiz
- `time.time()` orqali qancha vaqt ketganini o'lchayapmiz

## 2-qism: NumPy vektorlashgan hisoblash

```
In [95]: import time

n = 10_000_000
np_array = np.arange(n)

start = time.time()
result_np = np_array * 2
end = time.time()

print("NumPy vektorlashgan hisoblash vaqti:", end - start, "sekund")
```

NumPy vektorlashgan hisoblash vaqti: 0.01820206642150879 sekund

Bu yerda:

- `np_array` – NumPy massivi
- `np_array * 2` – **bitta yozuv bilan** barcha elementlarga operatsiya
- ichida C/Fortran darajasida optimallashtirilgan, SIMD va hokazo

---

## 3-qism: %timeit bilan solishtirish (Jupyter uchun maxsus)

Jupyter Notebook'da `timeit` "sehrli" (magic) komandasi bor, uni ishlatsak yanada aniqroq bo'ladi.

### Python ro'yxati:

```
In [96]: py_list = list(range(1_000_000))
%timeit [x * 2 for x in py_list]
```

37.8 ms ± 2.21 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

### NumPy vektorlashgan:

```
In [102... np_array = np.arange(1_000_000)
%timeit np_array * 2
```

1.45 ms ± 34.2 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

Natijada ko'rasizki:

- NumPy vektorlashgan usuli **bir necha baravar tezroq** chiqadi.

---

Quyida **list**, **tuple**, **dictionary** kabi Python ma'lumot turlarini qanday qilib **NumPy array**ga aylantirish va ularning qanday farq qilishi tushuntirilgan:

---

## 1. List → NumPy array

Python ro'yxatini (list) to'g'ridan-to'g'ri massivga aylantirish eng oson variant.

### ✓ Oddiy list

```
In [8]: lst = [1, 2, 3, 4]
arr = np.array(lst)
arr
```

```
Out[8]: array([1, 2, 3, 4])
```

### ✓ Ichma-ich list (2D list)

```
In [9]: lst2 = [[1, 2, 3],
               [4, 5, 6]]
```

```
arr2 = np.array(lst2)
arr2
```

```
Out[9]: array([[1, 2, 3],
               [4, 5, 6]])
```

**Natija:** 2×3 matritsa.

**Eslatma:** Har bir ichki ro'yxat uzunligi teng bo'lishi kerak, aks holda NumPy `dtype=object` bo'lgan "gibrid" massiv yaratadi.

---

## 2. Tuple → NumPy array

Tuple ham listga o'xshaydi, faqat o'zgarmas (immutable). NumPy uni ham 1×1 qabul qiladi:

✓ Oddiy tuple

```
In [10]: tpl = (10, 20, 30)
arr = np.array(tpl)
arr
```

```
Out[10]: array([10, 20, 30])
```

✓ Ichki tuple

```
In [11]: tpl2 = ((1, 2), (3, 4), (5, 6))
arr2 = np.array(tpl2)
arr2
```

```
Out[11]: array([[1, 2],
               [3, 4],
               [5, 6]])
```

NumPy buni 3×2 massiv deb oladi.

---

## 3. Dictionary → NumPy array

3 xil turi mavjud.

---

### 3.1. Faqat qiymatlar (values)ni arrayga aylantirish

Eng keng tarqalgan amaliy yechim:

```
In [12]: d = {"a": 1, "b": 2, "c": 3}
arr = np.array(list(d.values()))
arr
```

```
Out[12]: array([1, 2, 3])
```

---

### 3.2. Kalitlar (keys)ni arrayga aylantirish

```
In [13]: np.array(list(d.keys()))
```

```
Out[13]: array(['a', 'b', 'c'], dtype='<U1')
```

---

### 3.3. Dictionary elementlarini juftlik (key-value pair) sifatida array qilish

```
In [14]: arr = np.array(list(d.items()))
arr
```

```
Out[14]: array([[ 'a', '1'],
                [ 'b', '2'],
                [ 'c', '3']], dtype='<U21')
```

Bu massiv **2 ustunli** bo'ladi: key va value.

---

### Eslatma: Dictionary → np.array() bevosita berilsa nima bo'ladi?

```
In [15]: np.array({"a":1, "b":2})
```

```
Out[15]: array({'a': 1, 'b': 2}, dtype=object)
```

Bu **xato emas**, lekin foydasiz, chunki **object-type** massiv bo'ladi. Shuning uchun dictionaryni oldin **list** yoki **tuple** ko'rinishga o'tkazib olish kerak.

---

## 4. Aralash ma'lumotlar (mixed types)

Agar list, tuple yoki dict qiymatlari turli tipda bo'lsa:

```
In [16]: np.array([1, "salom", 3.5])
```

```
Out[16]: array(['1', 'salom', '3.5'], dtype='<U32')
```

NumPy **hammasini stringga** aylantiradi yoki `dtype=object` qiladi.

---

## Xulosa jadvali

| Python turi                | Arrayga aylantirish                     | Eslatma                   |
|----------------------------|-----------------------------------------|---------------------------|
| <b>List</b>                | <code>np.array(list)</code>             | Eng to'g'ridan-to'g'ri    |
| <b>Tuple</b>               | <code>np.array(tuple)</code>            | List bilan bir xil tarzda |
| <b>Dictionary (values)</b> | <code>np.array(list(d.values()))</code> | Eng ko'p ishlatiladi      |
| <b>Dictionary (keys)</b>   | <code>np.array(list(d.keys()))</code>   | Kalitlar massivini beradi |
| <b>Dictionary (items)</b>  | <code>np.array(list(d.items()))</code>  | 2D massiv hosil bo'ladi   |

---

## NumPy massiv metodlari — umumiy jadval

## Asosiy metodlar jadvali

| Kategoriya               | Funktsiya / metod                                                                   | Qisqa vazifasi                                                      |
|--------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Massiv yaratish          | <code>np.array</code>                                                               | Python list/tuple/dict dan massiv yaratish                          |
|                          | <code>np.zeros</code> , <code>np.ones</code> , <code>np.full</code>                 | 0, 1 yoki berilgan qiymatga to'ldirilgan massiv                     |
|                          | <code>np.arange</code> , <code>np.linspace</code>                                   | Ketma-ket sonlar yaratish                                           |
|                          | <code>np.eye</code>                                                                 | Diagonal bo'yicha 1 bo'lgan kvadrat matritsa                        |
|                          | <code>np.random.rand</code> , <code>np.random.randint</code>                        | Tasodifiy massivlar                                                 |
| Massiv haqida ma'lumot   | <code>.ndim</code> , <code>.shape</code> , <code>.size</code> , <code>.dtype</code> | O'lchamlar soni, shakli, elementlar soni, turi                      |
| Shakl o'zgartirish       | <code>reshape</code>                                                                | Massiv shaklini o'zgartirish                                        |
|                          | <code>ravel</code> , <code>flatten</code>                                           | Tekis (1D) ko'rinishga keltirish                                    |
|                          | <code>transpose</code> , <code>.T</code> , <code>swapaxes</code>                    | O'qlarni almashtirish                                               |
|                          | <code>expand_dims</code> , <code>squeeze</code>                                     | O'lcham qo'shish yoki siqish                                        |
| Tip o'zgartirish         | <code>astype</code>                                                                 | Elementlarning turini o'zgartirish (int → float va h.k.)            |
| Indekslash / tanlash     | <code>[]</code> slicing                                                             | Kesish: <code>a[1:5, :]</code>                                      |
|                          | Boolean indexing                                                                    | <code>a[a &gt; 0]</code> kabi shart bilan tanlash                   |
|                          | Fancy indexing                                                                      | <code>a[[0, 2, 5]]</code> bilan indekslar ro'yxati bo'yicha tanlash |
|                          | <code>np.where</code>                                                               | Shart bo'yicha indeks yoki qiymat tanlash                           |
| Arifmetik va ufunc       | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code> | Elementma-element arifmetika                                        |
|                          | <code>np.add</code> , <code>np.multiply</code>                                      | Ufunclar orqali arifmetika                                          |
|                          | <code>np.sqrt</code> , <code>np.exp</code> , <code>np.log</code>                    | Ilmiy funksiyalar                                                   |
|                          | <code>np.sin</code> , <code>np.cos</code> , <code>np.tan</code>                     | Trigonometrik funksiyalar                                           |
| Agregatsiya / statistika | <code>sum</code> , <code>mean</code> , <code>min</code> , <code>max</code>          | Yig'indi, o'rtacha, min/max                                         |
|                          | <code>std</code> , <code>var</code>                                                 | Standart og'ish, dispersiya                                         |
|                          | <code>argmin</code> , <code>argmax</code>                                           | Min/max indeksini qaytarish                                         |
|                          | <code>cumsum</code> , <code>cumprod</code>                                          | Kumulativ yig'indi, ko'paytma                                       |
| Saralash / qidirish      | <code>np.sort</code> , <code>.sort</code>                                           | Elementlarni tartiblash                                             |
|                          | <code>np.argsort</code>                                                             | Saralash tartibidagi indekslar                                      |
|                          | <code>np.unique</code>                                                              | Takrorlanmagan qiymatlar                                            |
|                          | <code>np.in1d</code> , <code>np.isin</code>                                         | Elementlar boshqa massivda bormi?                                   |

| Kategoriya       | Funktsiya / metod                                                 | Qisqa vazifasi                        |
|------------------|-------------------------------------------------------------------|---------------------------------------|
| Birlashtirish    | <code>np.concatenate</code>                                       | Massivlarni bir yo'nalishda ulash     |
|                  | <code>np.stack</code> , <code>vstack</code> , <code>hstack</code> | Yangi o'lcham bo'yicha ulash          |
|                  | <code>np.column_stack</code> , <code>row_stack</code>             | Ustun/satr bo'yicha ulash             |
| Bo'lish (split)  | <code>np.split</code>                                             | Massivni teng qismlarga bo'lish       |
|                  | <code>np.hsplit</code> , <code>np.vsplit</code>                   | Gorizontal/vertikal bo'lish           |
| Nusxa / view     | <code>.copy</code> , <code>np.copy</code>                         | To'liq nusxa olish                    |
|                  | <code>view</code>                                                 | Xotirani bo'lishadigan qarash (view)  |
| Chiziqli algebra | <code>np.dot</code> , <code>@</code> , <code>np.matmul</code>     | Skalyar ko'paytma, matritsa ko'paytma |
|                  | <code>np.linalg.inv</code>                                        | Matritsa teskari (inverse)            |
|                  | <code>np.linalg.det</code>                                        | Determinant                           |
|                  | <code>np.linalg.eig</code>                                        | O'z qiymat va o'z vektorlar           |

Endi har bir kategoriyaning batafsil, parametrlar va misollar bilan ko'rib chiqamiz.

## 1. Massiv yaratish metodlari

### 1.1. `np.array`

**Vazifasi:** Python list/tuple/np array'dan yangi NumPy massivi yaratadi.

`np.array(object, dtype=None, copy=True)`

- `object` – list, tuple, boshqa array va h.k.
- `dtype` – element turi (`np.int32`, `np.float64`, `np.bool_`, ...)
- `copy` – `True` bo'lsa, nusxa yaratadi; `False` bo'lsa, imkon bo'lsa, xotirani bo'lishadi.

**Misol:**

In [103...]

```
lst = [1, 2, 3]
a = np.array(lst)           # dtype avtomatik
b = np.array(lst, dtype=float) # float massiv

print(a, a.dtype)          # [1 2 3] int64 (masalan)
print(b, b.dtype)          # [1. 2. 3.] float64
```

```
[1 2 3] int64
[1. 2. 3.] float64
```

### 1.2. `np.zeros`, `np.ones`, `np.full`

**Vazifasi:** Berilgan shaklda 0, 1 yoki ma'lum qiymatga to'ldirilgan massiv yaratadi.



```
np.zeros(shape, dtype=float)
np.ones(shape, dtype=float)
np.full(shape, fill_value, dtype=None)
```

- `shape` – o'lcham: `3`, `(2,3)`, `(2,2,2)` va boshqalar
- `fill_value` – to'ldiriladigan qiymat
- `dtype` – element turi

#### Misol:

```
In [18]: z = np.zeros((2, 3))          # 2x3 nol
o = np.ones((2, 2), dtype=int)       # 2x2 birlik, int
f = np.full((1, 4), 7.5)            # 1x4, hamma element 7.5

print(z)
print(o)
print(f)

[[0.  0.  0.]
 [0.  0.  0.]]
[[1 1]
 [1 1]]
[[7.5 7.5 7.5 7.5]]
```

### 1.3. `np.arange`, `np.linspace`

#### `np.arange`

**Vazifasi:** `range` ga o'xshash, lekin array qaytaradi.

```
np.arange(start, stop, step, dtype=None)
```

- `start` – boshlanish (standart: 0)
- `stop` – to'xtash (kiritilmaydi)
- `step` – qadam (standart: 1)
- `dtype` – turi

#### Misol:

```
In [19]: a = np.arange(5)           # 0,1,2,3,4
b = np.arange(1, 10, 2)            # 1,3,5,7,9
print(a)
print(b)

[0 1 2 3 4]
[1 3 5 7 9]
```

#### `np.linspace`

**Vazifasi:** `[start, stop]` oraliqni berilgan `num` bo'lakka bo'lib, massiv qaytaradi.

```
np.linspace(start, stop, num=50, endpoint=True)
```

- `start` – boshlanish
- `stop` – oxiri

- `num` – elementlar soni
- `endpoint` – `True` bo'lsa, `stop` ham kiradi

**Misol:**

```
In [20]: x = np.linspace(0, 1, 5)
print(x)  # [0.  0.25 0.5  0.75 1.  ]

[0.  0.25 0.5  0.75 1.  ]
```

---

## 1.4. `np.eye`

**Vazifasi:** Diagonal bo'yicha 1 bo'lgan kvadrat / to'rtburchak matritsa.

`np.eye(N, M=None, k=0, dtype=float)`

- `N` – satrlar soni
- `M` – ustunlar soni (standart: `N`)
- `k` – qaysi diagonal: 0 – asosiy, 1 – yuqori, -1 – pastki
- `dtype` – element turi

**Misol:**

```
In [21]: I = np.eye(3)
print(I)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

J = np.eye(3, 4, k=1)
print(J)

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

---

## 1.5. Tasodifiy massivlar — `np.random.rand`, `np.random.randint`

`np.random.rand`

**Vazifasi:** [0, 1) oraliqda bir xil taqsimlangan tasodifiy float'lar.

`np.random.rand(d0, d1, ...)`

**Misol:**

```
In [26]: r = np.random.rand(2, 3)  # 2x3 tasodifiy son
print(r)

[[0.68624198 0.21453751 0.58373969]
 [0.53983553 0.69597384 0.75209571]]
```

```
np.random.randint
```

**Vazifasi:** [low, high) oraliqda tasodifiy integer'lar.

```
np.random.randint(low, high=None, size=None)
```

- `low` – pastki chegarasi (kiritiladi)
- `high` – yuqori chegarasi (kiritilmaydi)
- `size` – shakl (masalan `(2,3)` )

**Misol:**

```
In [ ]: ri = np.random.randint(0, 10, size=(2, 4))
        print(ri)
```

---

## 2. Massiv haqida ma'lumot metodlari

Bu metodlar array'ni "diagnostika qilish" uchun ishlatiladi.

```
a = np.arange(12).reshape(3, 4)
print(a)
```

### 2.1. `.ndim`, `.shape`, `.size`, `.dtype`

**Vazifasi:**

- `ndim` – o'lchamlar soni (1D, 2D, 3D...)
- `shape` – har bir o'lcham bo'yicha uzunlik
- `size` – jami elementlar soni
- `dtype` – elementlar turi

**Misol:**

```
In [27]: print("ndim:", a.ndim)      # 2
        print("shape:", a.shape)    # (3, 4)
        print("size:", a.size)      # 12
        print("dtype:", a.dtype)    # int64 va hokazo
```

```
ndim: 1
shape: (5,)
size: 5
dtype: int64
```

---

## 3. Shakl va o'lcham bilan ishlash

### 3.1. `reshape`

**Vazifasi:** Massiv shaklini (o'lchamini) o'zgartirish.

`a.reshape(newshape)`

- `newshape` – yangi shakl. Jami elementlar soni o'zgarmasligi shart.
- `-1` ishlatilsa, o'sha joy avtomatik hisoblanadi.

**Misol:**

```
In [28]: b = np.arange(12)
c = b.reshape(3, 4)
d = b.reshape(2, -1)  # 2 qator, ustunlarni o'zi topadi

print(c)
print(d)

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

---

## 3.2. `ravel` va `flatten`

**Ikkalasi ham tekis (1D) ko'rinishga keltiradi, farqi:**

- `ravel()` – imkon bo'lsa view qaytaradi (xotira bo'lishadi)
- `flatten()` – har doim nusxa (copy)

```
In [29]: b = np.arange(6).reshape(2, 3)
r = b.ravel()
f = b.flatten()

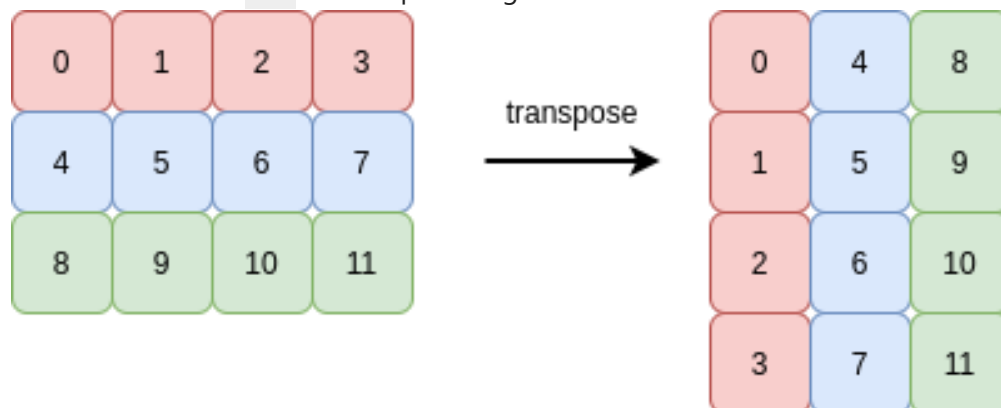
print(r)
print(f)

[0 1 2 3 4 5]
[0 1 2 3 4 5]
```

### 3.3. `transpose` va `.T`

**Vazifasi:** O'qlarni (axis) almashtirish.

2D matritsa uchun `.T` — transponirlangan matritsa.



```
In [71]: a = np.arange(6).reshape(2, 3)
print(a)
print(a.T)
```

```
[[0 1 2]
 [3 4 5]]
[[0 3]
 [1 4]
 [2 5]]
```

Ko'p o'lchamli massiv uchun:

```
In [72]: np.transpose(a, axes=(1, 0))
```

```
Out[72]: array([[0, 3],
               [1, 4],
               [2, 5]])
```

`.T` va `.transpose()` — ikkalasi ham **NumPy massivini transponatsiya qilish** uchun ishlatiladi.

Ammo ular **to'liq bir xil emas**. Farqi juda sodda:

#### 1. `.T` — faqat satr ↔ ustun almashtirish (2D uchun)

`.T` — faqat **massivning o'qlarini teskariga buradi** (reverse axes).

`a.T`

2D massivda `.T` → qator ↔ ustun almashadi. 3D yoki undan yuqori o'lchamda `.T` → barcha o'qlarni **teskari tartibga** o'tkazadi.

**Masalan 3D da:**

`(0,1,2)` → `.T` → `(2,1,0)`

#### 2. `.transpose()` — o'qlarni aniq tartib bilan berish imkoniyati bor

`.transpose()` funksiyasi orqali siz **istalgan o'qlarni istalgan tartibda** o'zgartirishingiz mumkin.

```
a.transpose(0, 2, 1)
```

Ya'ni siz indekslarni to'liq nazorat qilasiz.

---

`.T` da **parametr berib bo'lmaydi**, `.transpose()` da **parametr berish shart** (aks holda `.T` kabi ishlaydi).

---

## 2D MASSIV MISOL

```
import numpy as np
```

```
a = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
print(a.T)
```

```
print(a.transpose())
```

**Natija ikkala holatda ham bir xil:**

```
[[1 4]  
 [2 5]  
 [3 6]]
```

---

## 3D MASSIV MISOL (asosiy farq shu yerda bilinadi!)

```
b = np.arange(24).reshape(2, 3, 4)
```

```
b.T.shape
```

```
# (4, 3, 2)
```

```
b.transpose(0,2,1).shape
```

```
# (2, 4, 3)
```

```
b.transpose(2,0,1).shape
```

```
# (4, 2, 3)
```

`.T` → o'qlarni to'liq teskari tartibga o'tkazadi `.transpose()` → qaysi o'qlarni joyini o'zgartirishni **aniq belgilash** imkonini beradi.

---

## XULOSA

- 2D massivlarda `.T` va `.transpose()` bir xil.
  - 3D+ massivlarda `.T` → faqat teskari tartib (reverse axes).
  - `.transpose()` → o'qlarni xohlagancha boshqarasiz.
- 

## 3.4. swapaxes

**Vazifasi:** Ikki o'qning o'rnini almashtirish.

```
np.swapaxes(a, axis1, axis2)
```

yoki

```
a.swapaxes(axis1, axis2)
```

```
In [80]: np.swapaxes(a, 0, 1)
```

```
Out[80]: array([[0, 3],
               [1, 4],
               [2, 5]])
```

```
In [81]: a.swapaxes(0, 1)
```

```
Out[81]: array([[0, 3],
               [1, 4],
               [2, 5]])
```

---

## 3.5. expand\_dims va squeeze

`np.expand_dims`

**Vazifasi:** Yangi o'lcham qo'shish (1D → 2D kabi).

```
In [33]: x = np.array([1, 2, 3])
y = np.expand_dims(x, axis=0)
print(x.shape) # (3,)
print(y.shape) # (1, 3)
```

```
(3,)
(1, 3)
```

`np.squeeze`

**Vazifasi:** O'lchami 1 bo'lgan o'qlarni olib tashlaydi.

```
In [34]: z = np.array([[[[1, 2, 3]]]]) # shape (1,1,3)
print(z.shape)
zs = np.squeeze(z)
print(zs.shape) # (3,)
```

```
(1, 1, 3)
(3,)
```

---

## 4. Tip o'zgartirish — astype

**Vazifasi:** Elementlarning ma'lumot turini o'zgartirish.

```
a.astype(dtype)
```

- `dtype` – `np.float32`, `np.int64`, `'float'`, `'int'`, `'bool'` va h.k.

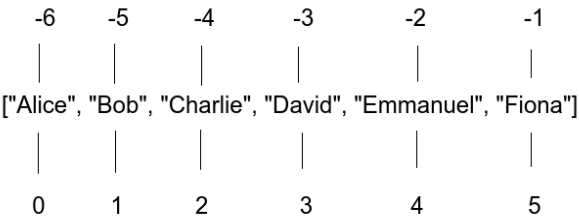
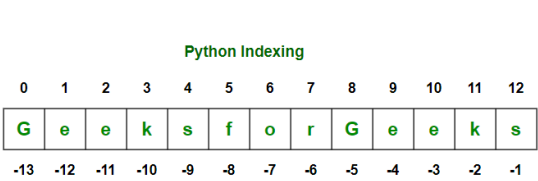
**Misol:**

```
In [35]: a = np.array([1.2, 3.7, 4.0])
b = a.astype(int)
c = a.astype(bool)

print(b, b.dtype) # [1 3 4] int64
print(c, c.dtype) # [ True  True  True] bool
```

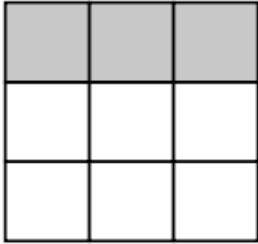
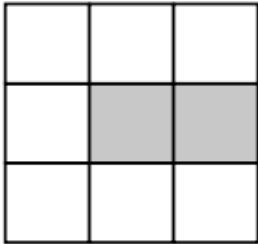
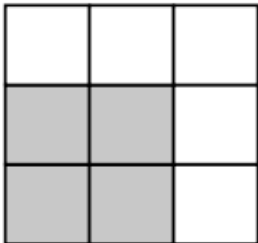
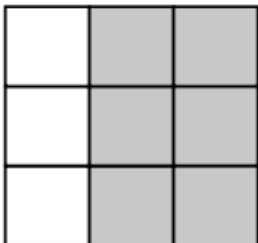
[1 3 4] int64  
[ True True True] bool

# 5. Indeksflash va tanlash



## 5.1. Oddiy slicing



|                                                                                     | Expression                  | Shape               |
|-------------------------------------------------------------------------------------|-----------------------------|---------------------|
|    | <code>arr2d[0]</code>       | <code>(3, )</code>  |
|                                                                                     | <code>arr2d[0, :]</code>    | <code>(3, )</code>  |
|                                                                                     | <code>arr2d[:1, :]</code>   | <code>(1, 3)</code> |
|    | <code>arr2d[1, 1:]</code>   | <code>(2, )</code>  |
|                                                                                     | <code>arr2d[1:2, 1:]</code> | <code>(1, 2)</code> |
|   | <code>arr2d[1:, :2]</code>  | <code>(2, 2)</code> |
|  | <code>arr2d[:, 1:]</code>   | <code>(3, 2)</code> |

```
In [36]: a = np.arange(10)
print(a[2:7])      # indeks 2..6
print(a[:, :2])    # har 2-ta element
```

```
[2 3 4 5 6]
```

```
[0 2 4 6 8]
```

2D massivda:

```
In [37]: b = np.arange(12).reshape(3,4)
print(b[1, 2])      # 2-qator, 3-ustun
print(b[0:2, 1:3])  # 1-2-qatorlar, 2-3-ustunlar
print(b[:, 0])      # barcha qatorlar, 1-ustun
```

```
6
```

```
[[1 2]
```

```
 [5 6]]
```

```
[0 4 8]
```

---

## 5.2. Boolean indexing

**Vazifasi:** Shart bo'yicha element tanlash.

**array > 5**

|       |       |       |       |       |      |      |      |      |      |
|-------|-------|-------|-------|-------|------|------|------|------|------|
| 1     | 2     | 3     | 4     | 5     | 6    | 7    | 8    | 9    | 10   |
| False | False | False | False | False | True | True | True | True | True |

```
In [38]: a = np.array([1, 2, 3, 4, 5])
mask = a > 3
print(mask)          # [False False False  True  True]
print(a[mask])       # [4 5]
```

```
[False False False  True  True]
[4 5]
```

To'g'ridan-to'g'ri:

```
In [39]: print(a[a % 2 == 0])  # juftlar
```

```
[2 4]
```

---

## 5.3. Fancy indexing

**Vazifasi:** Indekslar list/array'i bo'yicha element tanlash.

```
In [40]: a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(a[idx])  # [10 30 50]
```

```
[10 30 50]
```

2D misol:

```
In [41]: b = np.arange(12).reshape(3,4)
print(b[[0, 2], [1, 3]])  # (0,1) va (2,3) elementlar
```

```
[ 1 11]
```

---

## 5.4. np.where

### Vazifasi:

- 1-variant: Shart bo'yicha indekslar
- 2-variant: Shart bo'yicha qiymat tanlash (if-else)

```
In [42]: a = np.array([1, 2, 3, 4, 5])

idx = np.where(a > 3)
print(idx)          # (array([3, 4]),)

b = np.where(a > 3, a, 0)
print(b)            # [0 0 0 4 5]

(array([3, 4]),)
[0 0 0 4 5]
```

---

## 6. Arifmetik amallar va ufunc'lar

### 6.1. Elementma-element arifmetika

```
In [43]: a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)      # [11 22 33]
print(a - b)      # [-9 -18 -27]
print(a * b)      # [10 40 90]
print(b / a)      # [10. 10. 10.]
print(a ** 2)     # [1 4 9]

[11 22 33]
[-9 -18 -27]
[10 40 90]
[10. 10. 10.]
[1 4 9]
```

Broadcasting:

```
In [44]: print(a + 5)    # [6 7 8]

[6 7 8]
```

### 6.2. Ufunc: np.add, np.multiply va boshqalar

```
In [46]: print(np.add(a, b))      # [11 22 33]
print(np.multiply(a, 2))          # [2 4 6]
print(np.power(a, 3))             # [1 8 27]

[11 22 33]
[2 4 6]
[ 1  8 27]
```

---

## 6.3. Ilmiy funksiyalar: `sqrt`, `exp`, `log`

```
In [47]: x = np.array([1.0, 2.0, 4.0])
print(np.sqrt(x))      # ildiz
print(np.exp(x))       # e**x
print(np.log(x))       # natural log

[1.          1.41421356  2.          ]
[ 2.71828183  7.3890561  54.59815003]
[0.          0.69314718  1.38629436]
```

---

## 6.4. Trigonometrik funksiyalar

```
In [48]: t = np.array([0, np.pi/2, np.pi])
print(np.sin(t))
print(np.cos(t))
print(np.tan(t))

[0.0000000e+00  1.0000000e+00  1.2246468e-16]
[ 1.0000000e+00  6.123234e-17 -1.0000000e+00]
[ 0.00000000e+00  1.63312394e+16 -1.22464680e-16]
```

---

# 7. Agregatsiya va statistika

Massiv bo'yicha umumiy qiymatlar chiqarish.

## 7.1. `sum`, `mean`, `min`, `max`

```
In [49]: a = np.array([1, 2, 3, 4])

print(a.sum())      # 10
print(a.mean())     # 2.5
print(a.min())      # 1
print(a.max())      # 4
```

```
10
2.5
1
4
```

Axis bilan:

```
In [50]: b = np.arange(12).reshape(3, 4)

print(b.sum(axis=0)) # ustunlar bo'yicha
print(b.sum(axis=1)) # qatorlar bo'yicha
```

```
[12 15 18 21]
[ 6 22 38]
```

---

## 7.2. `std`, `var`

```
In [51]: print(a.std())    # standart og'ish
        print(a.var())    # dispersiya
```

```
1.118033988749895
1.25
```

---

## 7.3. argmin, argmax

**Vazifasi:** Min/max elementning indeksini qaytaradi.

```
In [52]: a = np.array([5, 2, 9, 1])
        print(a.argmin())    # 3 (qiymat 1)
        print(a.argmax())    # 2 (qiymat 9)
```

```
3
2
```

2D uchun `axis` bilan:

```
In [53]: b = np.arange(12).reshape(3,4)
        print(b.argmax(axis=1))    # har qatordagi eng katta element indeksi
```

```
[3 3 3]
```

---

## 7.4. cumsum, cumprod

```
In [54]: a = np.array([1, 2, 3, 4])

        print(a.cumsum())    # [1 3 6 10]
        print(a.cumprod())   # [1 2 6 24]
```

```
[ 1  3  6 10]
[ 1  2  6 24]
```

---

# 8. Saralash va qidirish

## 8.1. np.sort va .sort

**np.sort(a)** – yangi saralangan massiv qaytaradi (copy). **a.sort()** – massivni o'zida saralaydi (in-place).

```
In [55]: a = np.array([3, 1, 4, 1, 5])
        print(np.sort(a))    # [1 1 3 4 5]
        a.sort()
        print(a)              # [1 1 3 4 5]
```

```
[1 1 3 4 5]
[1 1 3 4 5]
```

---

## 8.2. np.argsort

**Vazifasi:** Saralash uchun indekslar tartibini qaytaradi.

```
In [56]: a = np.array([30, 10, 20])
         idx = np.argsort(a)
         print(idx)      # [1 2 0]
         print(a[idx])   # [10 20 30]
```

```
[1 2 0]
[10 20 30]
```

---

## 8.3. np.unique

**Vazifasi:** Takrorlanmaydigan elementlar.

```
In [89]: a = np.array([1, 2, 2, 3, 3, 3, 3])
         u = np.unique(a)
         print(u)      # [1 2 3]
```

```
[1 2 3]
```

Parametrlar:

`np.unique(a, return_counts=True)`

- `return_counts=True` bo'lsa, har bir qiymat nechta ekanini ham qaytaradi.

```
In [90]: values, counts = np.unique(a, return_counts=True)
         print(values)  # [1 2 3]
         print(counts)  # [1 2 3]
```

```
[1 2 3]
[1 2 4]
```

---

# 9. Birlashtirish (concatenate, stack) va bo'lish (split)

## 9.1. np.concatenate

```
In [59]: a = np.array([1, 2, 3])
         b = np.array([4, 5])
         c = np.concatenate([a, b])
         print(c)      # [1 2 3 4 5]
```

```
[1 2 3 4 5]
```

2D misol:

```
In [60]: x = np.array([[1, 2],
                       [3, 4]])
         y = np.array([[5, 6]])

         print(np.concatenate([x, y], axis=0)) # qator bo'yicha
```

```
[[1 2]
 [3 4]
 [5 6]]
```

---

## 9.2. np.stack, vstack, hstack

```
In [61]: a = np.array([1, 2, 3])
        b = np.array([4, 5, 6])

        print(np.stack([a, b], axis=0)) # 2x3
        print(np.vstack([a, b]))       # xuddi shunga o'xshash
        print(np.hstack([a, b]))       # 1x6
```

```
[[1 2 3]
 [4 5 6]]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

---

## 9.3. np.split, hspllit, vsplit

```
In [62]: a = np.arange(10)
        left, right = np.split(a, 2)
        print(left, right) # [0..4], [5..9]
```

```
[0 1 2 3 4] [5 6 7 8 9]
```

2D:

```
In [63]: b = np.arange(16).reshape(4,4)
        top, bottom = np.vsplit(b, 2)
        left, right = np.hsplit(b, 2)
```

---

# 10. Nusxa va view — .copy, view

## 10.1. copy

```
In [64]: a = np.array([1, 2, 3])
        b = a # reference (xotira bo'lishadi)
        c = a.copy() # yangi nusxa

        a[0] = 99
        print(b) # [99 2 3]
        print(c) # [1 2 3]
```

```
[99 2 3]
[1 2 3]
```

`copy()` — alohida xotira.

`view()` — xuddi o'sha xotiraga boshqa qarash.

Ko'p hollarda `.copy()` bilan ishlash xavfsizroq.

---

## 11. Chiziqli algebra — `dot`, `@`, `np.linalg`

### 11.1. `dot` va `@`

```
In [65]: A = np.array([[1, 2],
                      [3, 4]])
          B = np.array([[5, 6],
                      [7, 8]])

          print(A.dot(B))
          print(A @ B)      # xuddi shu

[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

---

### 11.2. `np.linalg.inv`, `np.linalg.det`

```
In [66]: A = np.array([[1., 2.],
                      [3., 4.]])

          det = np.linalg.det(A)
          invA = np.linalg.inv(A)

          print("det:", det)
          print("inv:\n", invA)

det: -2.0000000000000004
inv:
[[-2.   1. ]
 [ 1.5 -0.5]]
```

---

### 11.3. `np.linalg.eig`

```
In [67]: w, v = np.linalg.eig(A)    # o'z qiymatlar va vektorlar
          print("eigenvalues:", w)
          print("eigenvectors:\n", v)

eigenvalues: [-0.37228132  5.37228132]
eigenvectors:
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

Quyida **Python o'zgaruvchi turlari** va **NumPy'dagi dtype turlari** jadval ko'rinishida keltirilgan. Shuningdek, NumPy'da `dtype` bilan ishlash bo'yicha ham aniq misollar berib o'tilgan.

---

## 1. Python o'zgaruvchilari turlari (Jadval)



| Python tipi | Izoh                       | Misol            |
|-------------|----------------------------|------------------|
| int         | Butun son                  | x = 10           |
| float       | O'nlik son                 | y = 3.14         |
| str         | Matn (string)              | s = "Hello"      |
| bool        | Mantiqiy qiymat            | True , False     |
| list        | O'zgaruvchan ro'yxat       | [1, 2, 3]        |
| tuple       | O'zgarmas ro'yxat          | (1, 2, 3)        |
| dict        | Lug'at (kalit–qiymat)      | {"a": 1, "b": 2} |
| set         | Unikal elementlar to'plami | {1, 2, 3}        |
| complex     | Murakkab son               | 3 + 2j           |

## 2. NumPy dtype turlari (Jadval)

NumPy'da **ma'lumotlar turlari juda aniq** (C tili standardiga yaqinlashgan), chunki massivlar bir xil turdan iborat bo'lishi shart.

| NumPy dtype | Tavsifi                 | Bayt         | Misol                       |
|-------------|-------------------------|--------------|-----------------------------|
| int8        | -128 ... 127            | 1 bayt       | np.array([1], dtype='int8') |
| int16       | -32768 ... 32767        | 2 bayt       | dtype='int16'               |
| int32       | Standard butun          | 4 bayt       | dtype='int32'               |
| int64       | Katta butun son         | 8 bayt       | dtype='int64'               |
| uint8       | 0 ... 255               | 1 bayt       | Rasm piksel qiymatlari      |
| float16     | Past aniqlik float      | 2 bayt       | Tez hisob-kitob uchun       |
| float32     | Standart float          | 4 bayt       | dtype='float32'             |
| float64     | Yuqori aniqlik float    | 8 bayt       | NumPy default turi          |
| complex64   | Murakkab son            | 8 bayt       | 1+2j                        |
| complex128  | Keng aniqlik murakkab   | 16 bayt      | Ilmiy hisob-kitoblar        |
| bool_       | Mantiqiy                | 1 bayt       | True / False                |
| str_        | Fiks uzunlikdagi string | O'zgaruvchan | "abc"                       |
| object_     | Ob'ektlar               | O'zgaruvchan | Python object turida        |

## 3. NumPy'da dtype bilan ishlash (batafsil)

### 3.1. Massiv yaratishda dtype berish

```
import numpy as np
```

```
a = np.array([1, 2, 3], dtype='int32')
print(a.dtype)
```

**Natija:**

```
int32
```

---

### 3.2. dtype o'zgartirish — `astype()`

```
b = np.array([1.5, 2.8, 3.1])
c = b.astype('int32')
print(c)
```

**Natija:**

```
[1 2 3]
```

(O'nliklar kesiladi.)

---

### 3.3. Kichik baytli turga o'tkazish — xotira tejaladi

```
img = np.zeros((1000, 1000), dtype='uint8')
print(img.nbytes) # xotira miqdori
```

---

### 3.4. dtype ni tekshirish

```
arr = np.array([10, 20, 30], dtype=np.int16)
print(arr.dtype)
```

---

### 3.5. dtype konversiya misollari

1) `int` → `float`

```
x = np.array([1, 2, 3], dtype='int32')
xf = x.astype('float32')
```

2) `float` → `bool`

```
np.array([0.0, 1.5, -2.3]).astype(bool)
```

**Natija:**

```
[False True True]
```

---

## 4. Python tipi va NumPy dtype o'rtasidagi moslik

| Python tipi        | NumPy dtype                                | Izoh                        |
|--------------------|--------------------------------------------|-----------------------------|
| <code>int</code>   | <code>int32</code> yoki <code>int64</code> | OS va arxitekturaga bog'liq |
| <code>float</code> | <code>float64</code>                       | NumPy default float         |
| <code>bool</code>  | <code>bool_</code>                         | NumPy boolean               |
| <code>str</code>   | <code>str_</code>                          | Fiks uzunlikdagi string     |

| Python tipi | NumPy dtype     | Izoh                                  |
|-------------|-----------------|---------------------------------------|
| complex     | complex128      | Default murakkab son                  |
| list        | → ndarray       | Element turlari aniqlanadi            |
| tuple       | → ndarray       | list bilan bir xil                    |
| dict        | dtype bo'lmaydi | Ob'ekt bo'lib qoladi ( dtype=object ) |

## 5. dtype = object (ogohlantirish)

Agar massiv ichida turli tipdagi qiymatlar bo'lsa:

```
a = np.array([1, "salom", 3.5])
print(a.dtype)
```

**Natija:**

```
object
```

Bu ilmiy hisob-kitoblarda **tavsiya etilmaydi**, sekin ishlaydi.

## NumPy'da Universal Funksiyalar (ufunc)

### Ufunc (Universal Function) nima?

**Ufunc** — bu **element darajasida** (element-wise) massivlarga ishlov beruvchi, juda tez ishlaydigan NumPy funksiyalari.

Ular:

- **C tilida yozilgan** → shuning uchun juda tez ishlaydi
- **Vektorlashtirilgan** → loop (for) ishlatmaydi
- Broadcasting qo'llaydi
- Elementlar bo'yicha parallel ishlaydi
- SIMD optimizatsiyalardan foydalanadi

**Ufunc qanday ko'rinadi?**

Har bir ufunc massivni **element-by-element** hisoblaydi:

```
np.sqrt([1, 4, 9]) → [1, 2, 3]
```

## Ufunclarning asosiy turlari

Universal funksiyalar ikki guruhga bo'linadi:

### 1 Unary ufunc — bitta massivga ishlov beradi

Misollar:

| Funksiya                                                              | Tavsif                    |
|-----------------------------------------------------------------------|---------------------------|
| <code>np.abs()</code>                                                 | modul qiymat              |
| <code>np.sqrt()</code>                                                | kvadrat ildiz             |
| <code>np.exp()</code>                                                 | $e^x$ funksiyasi          |
| <code>np.log()</code>                                                 | natural log               |
| <code>np.sin()</code> , <code>np.cos()</code> , <code>np.tan()</code> | trigonometrik funksiyalar |
| <code>np.ceil()</code>                                                | yuqoriga yaxlitlash       |
| <code>np.floor()</code>                                               | pastga yaxlitlash         |

**Misol:**

```
a = np.array([1, 4, 9])
np.sqrt(a) # [1. 2. 3.]
```

## 2 Binary ufunc — ikkita massivga ishlov beradi

| Funksiya                       | Tavsif                               |
|--------------------------------|--------------------------------------|
| <code>np.add(a, b)</code>      | elementlar bo'yicha qo'shish         |
| <code>np.subtract(a, b)</code> | ayirish                              |
| <code>np.multiply(a, b)</code> | ko'paytirish                         |
| <code>np.divide(a, b)</code>   | bo'lish                              |
| <code>np.power(a, b)</code>    | darajaga ko'tarish                   |
| <code>np.maximum(a, b)</code>  | har bir elementdan maksimalini olish |
| <code>np.minimum(a, b)</code>  | minimal qiymat                       |

Misol:

```
a = np.array([1,2,3])
b = np.array([4,5,6])
np.add(a, b) # [5 7 9]
```

## Nega ufunclar juda tez ishlaydi?

Sabablar:

1. **C tilida optimizatsiyalangan kod** ishlatiladi.
2. Python for-looplari ishlatilmaydi.
3. **Vectorization** mavjud.
4. **SIMD va BLAS** optimizatsiyalari.
5. Xotira bir marotaba o'qiladi → cache optimizatsiyasi.

Shuning uchun:

```
np.add(a, b)
```

qo'lda yozilgandan 50–100 baravar tez:

```
for i in range(len(a)):
    c[i] = a[i] + b[i]
```

---

## Ufunclarning qo'shimcha imkoniyatlari

Ufunclar oddiy funksiyalardan farqli ravishda, quyidagi imkoniyatlarga ega:

---

### 1. Broadcasting

Massivlarning shape mos kelmaydigan joylarda ham ishlaydi.

```
np.add([1,2,3], 10)
→ [11,12,13]
```

---

### 2. Out parametri – natijani mavjud massivga yozish

```
a = np.array([1,2,3])
b = np.zeros(3)
np.multiply(a, 10, out=b)
```

---

### 3. Where parametri – shartli hisoblash

```
a = np.array([1,2,3,4])
np.sqrt(a, where=a>2)
```

---

### 4. Reduce — elementlarni ketma-ket yig'ish

```
np.add.reduce([1,2,3,4]) # 10
```

---

### 5. Accumulate — oraliq natijalarni chiqarish

```
np.add.accumulate([1,2,3,4])
→ [1, 3, 6, 10]
```

---

### 6. Outer — to'liq kombinatsion matritsa

```
np.multiply.outer([1,2,3], [4,5])
→ [[4,5], [8,10], [12,15]]
```

---

## Ufunclarning asosiy ro'yxati (toifalar bo'yicha)

---

### Matematik funksiyalar

abs, negative, floor, ceil, trunc, modf, exp, exp2, log, log2, log10, sqrt

---

### Aritmetik funksiyalar

add, subtract, multiply, divide, floor\_divide, power

---

## Trigonometrik funksiyalar

sin, cos, tan, arcsin, arccos, arctan

---

## Bitwise funksiyalar

bitwise\_and, bitwise\_or, bitwise\_xor, left\_shift, right\_shift

---

## Comparison funksiyalar

greater, greater\_equal, less, less\_equal, equal, not\_equal

---

## Logik funksiyalar

logical\_and, logical\_or, logical\_not, logical\_xor

---

## Ufunc'larni yaratish ham mumkin — `np.frompyfunc` va `np.vectorize`

Agar sizning Python funksiyangiz bo'lsa, uni vektorlashtirib ufuncga aylantirish mumkin.

```
def kv(x):  
    return x*x
```

```
uf = np.frompyfunc(kv, 1, 1)
```

```
uf(np.array([1,2,3]))
```

---

## XULOSA

NumPy'da universal funksiyalar:

- **massivlar ustida element-wise ishlaydi**
- **juda tez**, chunki C tilida optimizatsiyalangan
- **broadcasting** qo'llaydi
- `reduce`, `accumulate`, `outer` kabi **kengaytirilgan funksionalga ega**

Ular **NumPy'ning eng kuchli mexanizmi** bo'lib, mashinali o'qitish, tasvirlarni qayta ishlash, matematik modelashtirishda asosiy rol o'ynaydi.

---



